PROCESS SYNCHRONIZATION

مقایسهٔ جامع الگوریتم‌های همگام‌سازی فرایندها

دکر • پترسون • سمافور • مانیتور • خوانندگان و نویسندگان

مشخصات

عنوان مقاله	مقایسهٔ جامع الگوریتم‌های دکر، پترسون، سمافور، مانیتور و خوانندگان–نویسندگان
نام درس	سیستم های نهفته
نویسنده	نجلا داورپناه
استاد	مجید شمشیرگران
دانشگاه / دانشکده	دانشگاه اشراق
تاریخ	۱۹ آبان ۱۴۰۵ (۱۰ November ۲۰۲۶)

نسخهٔ فارسی

چکیده و فهرست

هنگامی که چند فرایند همزمان به داده‌های مشترک دسترسی دارند، تنها یک ابزار همگام‌سازی درست می‌تواند از ناسازگاری داده جلوگیری کند. این مقاله پنج روش شناخته‌شده را بررسی و مقایسه می‌کند: دو الگوریتم نرم‌افزاری **دکر** و **پترسون** برای دو فرایند، ابزار **سمافور** در سطح سیستم‌عامل، سازهٔ **مانیتور** در سطح زبان برنامه‌نویسی، و مسئلهٔ کلاسیک **خوانندگان و نویسندگان**. برای هر روش سودوکد، کد C، فلوچارت، ردگیری اجرا، مثال و نمونه‌سؤال ارائه شده است. در پایان، جدول‌های مقایسه (انحصار متقابل، پیشرفت، انتظار محدود، قحطی، بن‌بست، انتظار مشغول، مقیاس‌پذیری و ...) و نمودارها آمده‌اند.

کلیدواژه‌ها: همگام‌سازی، ناحیهٔ بحرانی، انحصار متقابل، دکر، پترسون، سمافور، مانیتور، خوانندگان و نویسندگان

فهرست مطالب

۱	مقدمه و مسئلهٔ ناحیهٔ بحرانی
۲	الگوریتم دکر
۳	الگوریتم پترسون
۴	سمافور
۵	مانیتور
۶	خوانندگان و نویسندگان
۷	مقایسهٔ جامع و جدول‌ها
۸	نمودارها
۹	مثال‌ها و نمونه‌سؤال‌ها
۱۰	نتیجه‌گیری
۱۱	منابع

۱. مقدمه و مسئلهٔ ناحیهٔ بحرانی

در سیستم‌های چندفرایندی و چندنخی، فرایندها برای تبادل اطلاعات یا استفاده از منابع، داده‌های مشترک را می‌خوانند و می‌نویسند. اگر دسترسی همزمان به این داده‌ها کنترل نشود، نتیجهٔ اجرا به ترتیب دقیق درهم‌آمیختگی دستورها بستگی پیدا می‌کند؛ پدیده‌ای که **شرایط رقابتی (Race Condition)** نام دارد.

مثال کلاسیک: دو فرایند `P1` و `P2` روی متغیر مشترک `counter = 5` به‌ترتیب `counter++` و `counter--` را اجرا می‌کنند. هر یک از این دستورها در سطح ماشین سه گام دارد: بارگذاری در ثبات، تغییر ثبات و ذخیره در حافظه. بسته به ترتیب درهم‌آمیختگی گام‌ها، مقدار نهایی می‌تواند ۴، ۵ یا ۶ باشد، در حالی که تنها ۵ درست است.

ساختار ناحیهٔ بحرانی

بخشی از کد که به داده مشترک دست می‌زند **ناحیهٔ بحرانی (Critical Section)** نام دارد. هر فرایند چهار بخش دارد: ورود (Entry)، ناحیهٔ بحرانی، خروج (Exit) و باقیمانده (Remainder). الگوریتم‌های این مقاله در حقیقت روش‌های گوناگونی برای ساختن بخش‌های ورود و خروج‌اند.

سودوکد — general structure

```
do {
    ENTRY SECTION        // ask for permission
        CRITICAL SECTION // touches shared data
    EXIT SECTION         // give permission to others
        REMAINDER SECTION
} while (true);
```

سه شرط درستی

- **انحصار متقابل (Mutual Exclusion):** در هر لحظه حداکثر یک فرایند درون ناحیهٔ بحرانی باشد.
- **پیشرفت (Progress):** اگر ناحیهٔ بحرانی خالی است و فرایندهایی خواهان ورودند، تصمیم دربارهٔ ورود فقط میان همین متقاضیان و در زمان محدود گرفته شود؛ فرایندی که در بخش باقیمانده است نباید مانع شود.
- **انتظار محدود (Bounded Waiting):** از لحظهٔ درخواست یک فرایند تا ورود او، تعداد دفعاتی که دیگران پیش از او وارد می‌شوند محدود باشد (جلوگیری از قحطی).

فرض‌ها: دربارهٔ سرعت نسبی فرایندها هیچ فرضی نداریم و خواندن یا نوشتن یک متغیر ساده را عمل یکپارچه (Atomic) می‌دانیم.

دسته‌بندی روش‌های این مقاله

جدول ۱ — دسته‌بندی روش‌ها

روش	نوع	سطح پیاده‌سازی	ابزار مورد نیاز
Dekker	الگوریتم نرم‌افزاری	برنامهٔ کاربر	فقط حافظهٔ مشترک
Peterson	الگوریتم نرم‌افزاری	برنامهٔ کاربر	فقط حافظهٔ مشترک
Semaphore	ابزار همگام‌سازی	هستهٔ سیستم‌عامل / کتابخانه	دستورهای اتمیک و صف انتظار
Monitor	سازهٔ ساخت‌یافته	زبان برنامه‌نویسی / کتابخانه	قفل + متغیر شرطی
Readers–Writers	مسئلهٔ کلاسیک	الگوی طراحی	حل با سمافور یا مانیتور

نکته

«خوانندگان و نویسندگان» یک **مسئله** است، نه یک ابزار. در جدول‌های مقایسه، منظور از این ستون، راه‌حل استاندارد سمافوری (اولویت با خواننده) است و تفاوت راه‌حل‌های دیگر جداگانه ذکر می‌شود.

۲. الگوریتم دکر

الگوریتم دکر نخستین راه‌حل نرم‌افزاری درست برای مسئلهٔ ناحیهٔ بحرانی میان **دو** فرایند است. این الگوریتم را ریاضی‌دان هلندی تئودوروس دکر (T. J. Dekker) ابداع کرد و دایکسترا (Dijkstra) آن را در سال ۱۹۶۵ منتشر کرد. تنها از حافظهٔ مشترک استفاده می‌کند و به دستور سخت‌افزاری ویژه‌ای نیاز ندارد.

ایده

دو متغیر مشترک داریم: آرایهٔ `flag[2]` که نشان می‌دهد هر فرایند خواهان ورود است، و متغیر `turn` که در تعارض تعیین می‌کند اولویت با کیست. هر فرایند ابتدا علاقه‌اش را اعلام می‌کند؛ اگر رقیب هم خواهان بود، اگر نوبت با او نیست، **عقب‌نشینی** می‌کند (پرچمش را پایین می‌آورد)، تا نوبتش شود صبر می‌کند و دوباره تلاش می‌کند. عقب‌نشینی همان چیزی است که بن‌بست را در روش‌های ساده‌تر برطرف می‌کند.

سودوکد — process Pi (j = ۱ - i)

```
shared: boolean flag[2] = {false, false};
        int turn = 0;


Pi:
  repeat
    flag[i] = true;
    while (flag[j] == true) {
        if (turn != i) {            // not my turn: back off
            flag[i] = false;
            while (turn != i) ;     // wait for my turn
            flag[i] = true;
        }
    }
    // ---- CRITICAL SECTION ----
    turn = j;                       // hand priority to the other
    flag[i] = false;
    // ---- REMAINDER SECTION ----
  forever
```

کد C (C۱۱ atomics)

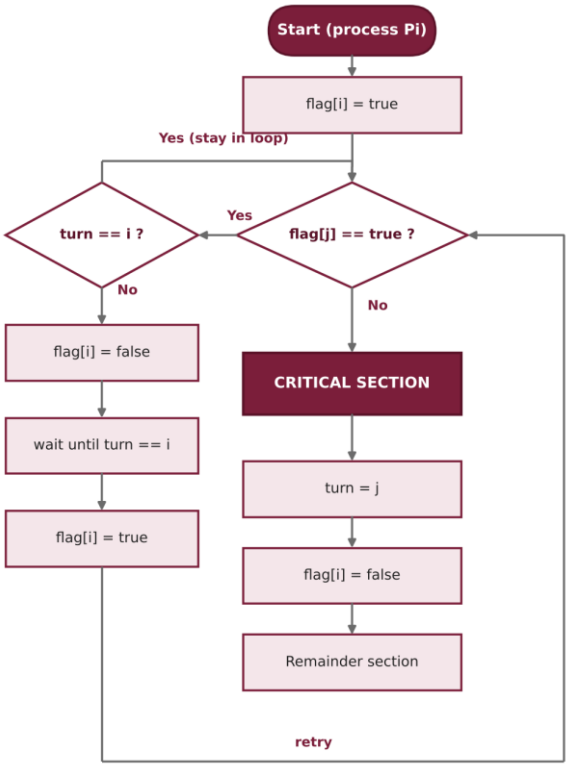
```
#include <stdatomic.h>
#include <stdbool.h>

atomic_bool flag[2] = { false, false };
atomic_int  turn    = 0;

void dekker_enter(int i) {
    int j = 1 - i;
    atomic_store(&flag[i], true);
    while (atomic_load(&flag[j])) {
        if (atomic_load(&turn) != i) {
            atomic_store(&flag[i], false);
            while (atomic_load(&turn) != i) { /* busy wait */ }
            atomic_store(&flag[i], true);
        }
    }
}
```

```
void dekker_exit(int i) {
    atomic_store(&turn, 1 - i);
    atomic_store(&flag[i], false);
}
```

در کد بالا از متغیرهای اتمیک با ترتیب حافظهٔ پیش‌فرض (`seq_cst`) استفاده شده است. روی پردازنده‌های امروزی، اگر به‌جای آن‌ها متغیرهای معمولی به کار رود، بازچینی دستورات حافظه می‌تواند الگوریتم را بشکند.



شکل ۱ — فلوچارت الگوریتم دکر برای فرایند Pi

ردگیری اجرا

فرض کنید `turn = 0` و هر دو فرایند همزمان بخواهند وارد شوند:

جدول ۲ — ردگیری دکر (هر دو فرایند متقاضی)

گام	P۰	P۱	f[۰]	f[۱]	turn
۱	flag[۰]=true	flag[۱]=true	T	T	۰
۲	flag[۱] درست است؛ turn==۰ → در حلقه می‌ماند	turn≠۱ → flag[۰] درست است؛ flag[۱]=false	T	F	۰
۳	flag[۱] نادرست → وارد ناحیهٔ بحرانی	منتظر turn==۱	T	F	۰
۴	turn=۱ ؛ flag[۰]=false (خروج)	turn==۱ → flag[۱]=true	F	T	۱
۵	بخش باقیمانده	flag[۰] نادرست → وارد ناحیهٔ بحرانی	F	T	۱

ویژگی‌ها

- **انحصار متقابل:** برقرار است؛ ورود فقط وقتی ممکن است که `flag[j]` نادرست دیده شود.
- **پیشرفت:** برقرار است؛ اگر رقیب خواهان نیست، `flag[j]=false` است و فرایند بدون توجه به `turn` وارد می‌شود.
- **انتظار محدود:** برقرار است؛ پس از خروج، `turn` به رقیب داده می‌شود، پس هر فرایند حداکثر یک بار منتظر ورود رقیب می‌ماند.
- **معایب:** فقط دو فرایند؛ انتظار مشغول (Busy Waiting)؛ پیچیدگی منطق؛ نیاز به ترتیب‌دهی حافظه در سخت‌افزار مدرن.

۳. الگوریتم پترسون

گری پترسون (G. L. Peterson) در سال ۱۹۸۱ راه‌حلی بسیار ساده‌تر از دکر برای دو فرایند ارائه کرد. متغیرها همان‌هاست (`flag[2]` و `turn`)، اما منطق «تعارف» به‌کار می‌رود: فرایندی که می‌خواهد وارد شود، ابتدا علاقه‌اش را اعلام می‌کند و سپس **نوبت را به رقیب می‌دهد**. اگر هر دو همزمان بخواهند، آن‌که آخرین بار `turn` را نوشته نوبت را به دیگری داده و منتظر می‌ماند.

سودوکد — process Pi (j = ۱ - i)

```
shared: boolean flag[2] = {false, false};
        int turn;


Pi:
  repeat
    flag[i] = true;                     // I am interested
    turn = j;                           // but you go first
    while (flag[j] == true && turn == j) ;   // wait
    // ---- CRITICAL SECTION ----
    flag[i] = false;                    // I am done
    // ---- REMAINDER SECTION ----
  forever
```

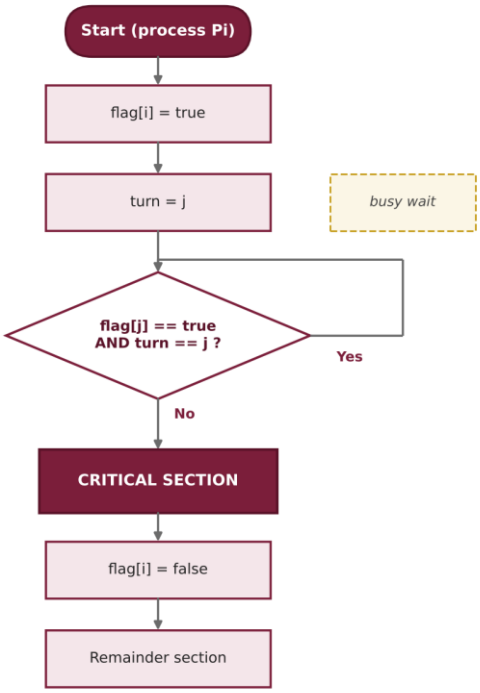
کد C (C۱۱ atomics)

```c
#include <stdatomic.h>
#include <stdbool.h>

atomic_bool flag[2] = { false, false };
atomic_int  turn;

void peterson_enter(int i) {
    int j = 1 - i;
    atomic_store(&flag[i], true);
    atomic_store(&turn, j);
    while (atomic_load(&flag[j]) && atomic_load(&turn) == j) {
        /* busy wait */
    }
}

void peterson_exit(int i) {
    atomic_store(&flag[i], false);
}
```



شکل ۲ — فلوچارت الگوریتم پترسون

اثبات کوتاه انحصار متقابل

فرض کنید `P0` و `P1` هر دو در ناحیهٔ بحرانی باشند. هر کدام پیش از ورود شرط حلقه را نادرست دیده است. `P0` یا `flag[1]` را نادرست دیده یا `turn==1` را؛ اما `P1` پیش از آن `flag[1]=true` را نوشته و تا خروج آن را نگه می‌دارد، پس `P0` باید `turn!=1` یعنی `turn==0` دیده باشد. به‌همین ترتیب `P1` باید `turn==1` دیده باشد. چون `turn` یک متغیر واحد است و آخرین نویسنده تعیین‌کننده است، هر دو شرط هم‌زمان ممکن نیست؛ تناقض.

ردگیری اجرا

جدول ۳ — ردگیری پترسون (هر دو متقاضی)

گام	P۰	P۱	f[۰]	f[۱]	turn
۱	flag[۰]=true	—	T	F	?
۲	—	flag[۱]=true	T	T	?
۳	turn=۱	—	T	T	۱
٤	—	turn=۰ (آخرین نویسنده)	T	T	۰
٥	flag[۱]&&turn==۱ ؟ نادرست ← وارد می‌شود	flag[۰]&&turn==۰ ؟ درست ← منتظر	T	T	۰
٦	flag[۰]=false (خروج)	شرط نادرست شد ← وارد می‌شود	F	T	۰

ویژگی‌ها و نکات

- **انحصار متقابل، پیشرفت و انتظار محدود** هر سه برقرارند؛ رقیب حداکثر یک بار پیش از شما وارد می‌شود.

- **معایب:** فقط دو فرایند، انتظار مشغول، و نیاز به ترتیب حافظهٔ ترتیبی؛ بدون `seq_cst` یا حصار حافظه، بافر نوشتن (Store Buffer) می‌تواند ترتیب `flag[i]=true` و خواندن `flag[j]` را برگرداند و هر دو فرایند را وارد کند.
- تعمیم برای `N` فرایند با **قفل فیلتر (Filter Lock)** یا الگوریتم نانوایی (Bakery) ممکن است، اما هزینه و پیچیدگی بیشتری دارد.

۴. سمافور

سمافور را دایکسترا در ۱۹۶۵ معرفی کرد. سمافور یک متغیر صحیح `S` است که تنها از طریق دو عمل **اتمیک** `wait` (که دایکسترا P یا proberen نامید) و `signal` (یا V یا verhogen) تغییر می‌کند. برخلاف الگوریتم‌های دکر و پترسون، سمافور برای **N فرایند** کار می‌کند و در پیاده‌سازی «مسدودکننده» به‌جای انتظار مشغول، فرایند را به صف انتظار می‌برد و پردازنده را آزاد می‌کند.

تعریف و پیاده‌سازی مسدودکننده

سودوکد — semaphore operations

```
typedef struct { int value; queue list; } semaphore;

wait(S):                      // P, down  (executed atomically)
    S.value = S.value - 1;
    if (S.value < 0) {
        add this process to S.list;
        block();              // sleep: no busy waiting
    }

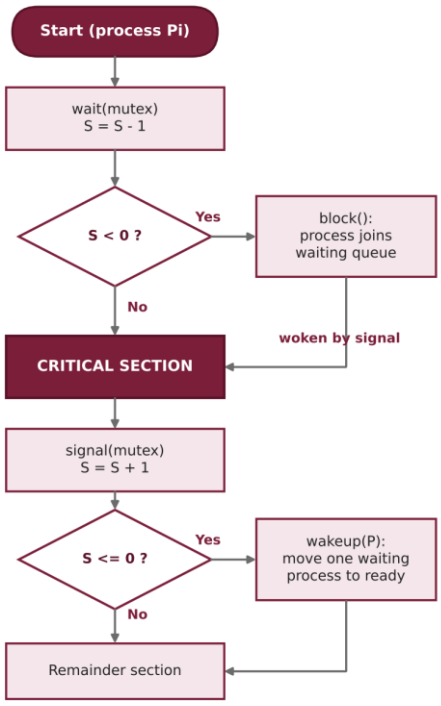
signal(S):                    // V, up    (executed atomically)
    S.value = S.value + 1;
    if (S.value <= 0) {
        remove a process P from S.list;
        wakeup(P);
    }
```

در این پیاده‌سازی، مقدار منفی `S` برابر تعداد فرایندهای منتظر است. دو نوع رایج: **سمافور دودویی (Binary / Mutex)** با مقدار ۰ یا ۱ برای انحصار متقابل، و **سمافور شمارنده (Counting)** برای کنترل منابع چندنمونه‌ای.

سودوکد — mutual exclusion

```
semaphore mutex = 1;

Pi:
  repeat
    wait(mutex);
      // ---- CRITICAL SECTION ----
    signal(mutex);
      // ---- REMAINDER SECTION ----
  forever
```



شکل ۳ — فلوچارت کار با سمافور (wait / signal)

مثال: تولیدکننده–مصرف‌کننده با بافر محدود

سه سمافور لازم است: `empty` (تعداد خانه‌های خالی، مقدار اولیه `N`)، `full` (تعداد خانه‌های پر، مقدار اولیه ۰) و `mutex` (انحصار روی بافر).

سودوکد

```
semaphore mutex = 1, empty = N, full = 0;

Producer:                          Consumer:
  repeat                             repeat
    item = produce();                  wait(full);
    wait(empty);                       wait(mutex);
    wait(mutex);                       item = remove();
    insert(item);                      signal(mutex);
    signal(mutex);                     signal(empty);
    signal(full);                      consume(item);
  forever                            forever
```

کد C (POSIX threads + semaphores)

```c
#include <pthread.h>
#include <semaphore.h>
#define N 8

int buf[N], in = 0, out = 0;
sem_t empty, full, mutex;

void *producer(void *arg) {
    for (int i = 0; i < 100; i++) {
        sem_wait(&empty);
        sem_wait(&mutex);
```

```
        buf[in] = i;  in = (in + 1) % N;
        sem_post(&mutex);
        sem_post(&full);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 100; i++) {
        sem_wait(&full);
        sem_wait(&mutex);
        int x = buf[out];  out = (out + 1) % N;
        sem_post(&mutex);
        sem_post(&empty);
        (void)x;                /* use the item here */
    }
    return NULL;
}

int main(void) {
    pthread_t p, c;
    sem_init(&empty, 0, N);
    sem_init(&full,  0, 0);
    sem_init(&mutex, 0, 1);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    return 0;
}
```

ویژگی‌ها و خطرها

- **انحصار متقابل و پیشرفت** در استفادهٔ درست برقرارند. **انتظار محدود** به سیاست صف بستگی دارد: سمافور قوی (صف FIFO) آن را تضمین می‌کند، سمافور ضعیف (بیدار کردن دلخواه) نه.
- **بن‌بست:** ترتیب اشتباه عمل‌ها، مثلاً `wait(mutex)` پیش از `wait(empty)` در تولیدکننده، می‌تواند بن‌بست بسازد.
- **خطاپذیری بالا:** فراموش‌کردن `signal` یا جابه‌جایی `wait` و `signal`، انحصار را می‌شکند یا فرایندها را برای همیشه مسدود می‌کند؛ ابزار سطح پایین و بدون ساختار است.
- **وارونگی اولویت (Priority Inversion)** ممکن است رخ دهد و نیاز به پروتکل‌هایی مانند وراثت اولویت دارد.

۵. مانیتور

مانیتور را هوآر (C. A. R. Hoare، ۱۹۷۴) و برینچ هانسن (Brinch Hansen) برای رفع خطاپذیری سمافور معرفی کردند. مانیتور یک **نوع داده انتزاعی** است که داده‌های مشترک و رویه‌های دسترسی به آن‌ها را در خود دارد و **کامپایلر یا زبان** تضمین می‌کند که در هر لحظه حداکثر یک فرایند درون رویه‌های آن فعال باشد؛ بنابراین برنامه‌نویس نیازی به نوشتن `wait/signal` برای انحصار متقابل ندارد.

برای همگام‌سازی شرطی (مثلاً «بافر خالی نیست») از **متغیر شرطی (Condition Variable)** استفاده می‌شود که دو عمل دارد: `x.wait()` فرایند را مسدود و قفل مانیتور را آزاد می‌کند، و `x.signal()` یکی از منتظران را بیدار می‌کند. برخلاف `signal` سمافور، اگر هیچ‌کس منتظر نباشد، `x.signal()` **اثری ندارد** و به یاد سپرده نمی‌شود.

دو معناشناسی signal

- **هوآر (Signal-and-Wait):** فرایند سیگنال‌دهنده بلافاصله مانیتور را به بیدارشده می‌دهد؛ شرط تضمین‌شده درست است و `if` کافی است.
- **مزا (Signal-and-Continue):** سیگنال‌دهنده ادامه می‌دهد و بیدارشده بعداً قفل را می‌گیرد؛ ممکن است شرط تا آن زمان تغییر کند، پس باید از **`while`** استفاده کرد. جاوا، pthreads و اغلب سیستم‌های واقعی از مزا پیروی می‌کنند.

سودوکد — bounded-buffer monitor

```
monitor BoundedBuffer {
    int buf[N];  int count = 0, in = 0, out = 0;
    condition notFull, notEmpty;

    procedure put(x) {
        while (count == N) notFull.wait();
        buf[in] = x;  in = (in + 1) % N;  count++;
        notEmpty.signal();
    }

    procedure get() {
        while (count == 0) notEmpty.wait();
        x = buf[out]; out = (out + 1) % N; count--;
        notFull.signal();
        return x;
    }
}   // at most one process is active inside at any time
```

کد C (pthread mutex + condition variables)

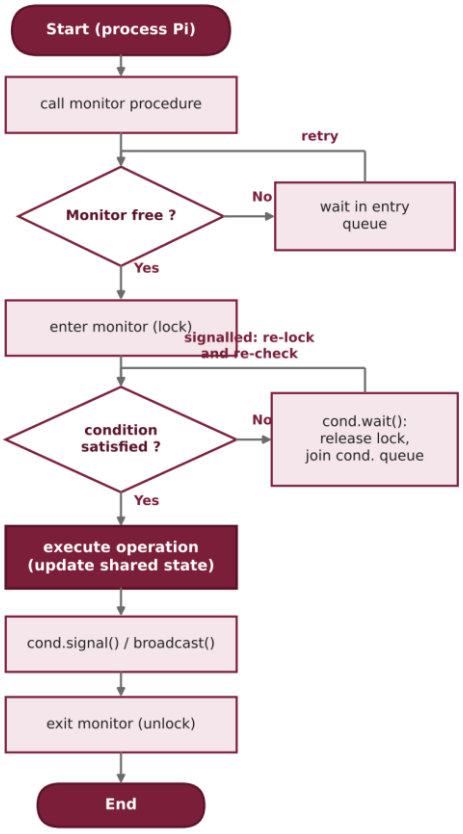
```
#include <pthread.h>
#define N 8

static int buf[N], in = 0, out = 0, count = 0;
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;  /* monitor lock */
static pthread_cond_t notFull  = PTHREAD_COND_INITIALIZER;
static pthread_cond_t notEmpty = PTHREAD_COND_INITIALIZER;

void put(int x) {
    pthread_mutex_lock(&m);                      /* enter monitor */
    while (count == N) pthread_cond_wait(&notFull, &m);
    buf[in] = x;  in = (in + 1) % N;  count++;
    pthread_cond_signal(&notEmpty);
    pthread_mutex_unlock(&m);                    /* exit monitor  */
}
```

```
int get(void) {
    pthread_mutex_lock(&m);
    while (count == 0) pthread_cond_wait(&notEmpty, &m);
    int x = buf[out];  out = (out + 1) % N;  count--;
    pthread_cond_signal(&notFull);
    pthread_mutex_unlock(&m);
    return x;
}
```

در جاوا همین الگو با `synchronized` و `wait()/notifyAll()` نوشته می‌شود. C فاقد مانیتور داخلی است و در اینجا مانیتور با قفل و متغیر شرطی **شبیه‌سازی** شده است.



شکل ۴ — فلوچارت ورود و خروج از مانیتور با متغیر شرطی

ویژگی‌ها و خطرها

- **انحصار متقابل خودکار** و ساخت‌یافته؛ ایمن‌ترین و خواناترین ابزار این مقاله.
- **بن‌بست** هنوز ممکن است، مثلاً با فراخوانی تو در توی مانیتورها (Nested Monitor Call) که قفل دیگری را نگه می‌دارند.
- **قحطی** در مزا ممکن است اگر بیدارشده‌ها بارها شرط را ناکافی ببینند و صف عادلانه نباشد.
- نیازمند پشتیبانی زبان یا کتابخانه؛ هزینهٔ قفل و تعویض متن دارد.

۶. خوانندگان و نویسندگان

مسئلهٔ خوانندگان و نویسندگان را کورتوآ، هیمنس و پارناس (Courtois, Heymans, Parnas) در ۱۹۷۱ مطرح کردند. یک داده مشترک (مثلاً پایگاه داده) داریم؛ **خواننده‌ها** فقط می‌خوانند و می‌توانند همزمان باشند، اما **نویسنده** باید دسترسی کاملاً انحصاری داشته باشد: نه نویسندهٔ دیگری و نه هیچ خواننده‌ای همزمان با او نباشد. برخلاف انحصار متقابل ساده، اینجا هم‌روندی میان خواننده‌ها مجاز است و بازده را بالا می‌برد.

سه نسخهٔ مسئله

جدول ۴ — نسخه‌های مسئلهٔ خوانندگان–نویسندگان

نسخه	قاعده	پیامد
اول — اولویت با خواننده	تا وقتی خواننده‌ای در حال خواندن است، خوانندهٔ جدید مجاز است وارد شود	احتمال **قحطی نویسنده**
دوم — اولویت با نویسنده	وقتی نویسنده‌ای منتظر است، خوانندهٔ جدید وارد نمی‌شود	احتمال **قحطی خواننده**
سوم — عادلانه	ورود به‌ترتیب نوبت (FIFO) برای همهٔ درخواست‌ها	بدون قحطی؛ هم‌روندی کمتر

راه‌حل با سمافور (اولویت با خواننده)

`rw_mutex` دسترسی به داده را کنترل می‌کند، `read_count` تعداد خواننده‌های فعال را نگه می‌دارد و `mutex` از خود `read_count` محافظت می‌کند. نخستین خواننده نویسنده‌ها را بیرون می‌اندازد و آخرین خواننده دوباره اجازه می‌دهد.

سودوکد

```
semaphore rw_mutex = 1, mutex = 1;
int read_count = 0;

Writer:                          Reader:
  wait(rw_mutex);                  wait(mutex);
    // WRITE                       read_count++;
  signal(rw_mutex);                if (read_count == 1) wait(rw_mutex);
                                   signal(mutex);
                                     // READ
                                   wait(mutex);
                                   read_count--;
                                   if (read_count == 0) signal(rw_mutex);
                                   signal(mutex);
```

کد C (POSIX semaphores)

```c
#include <semaphore.h>

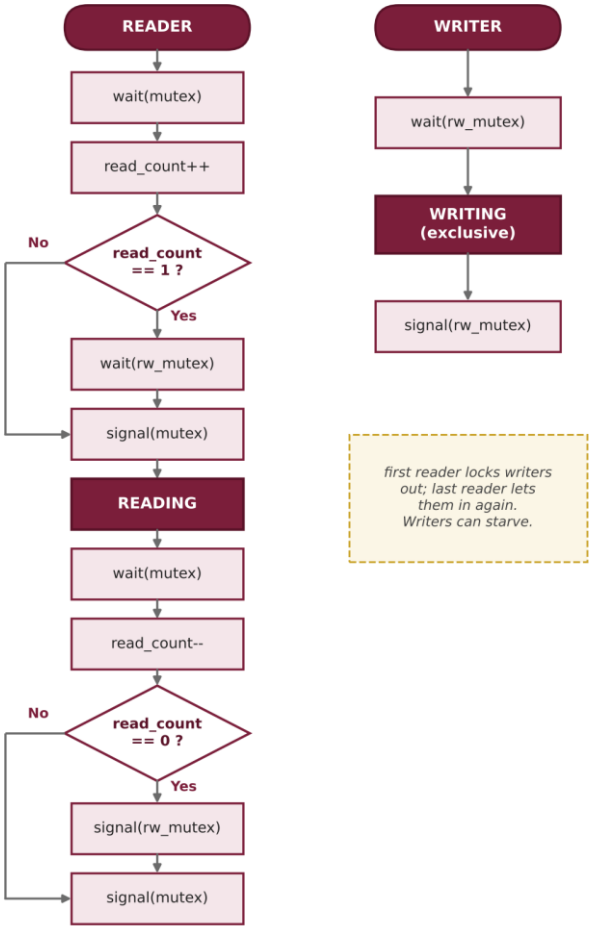
sem_t rw_mutex, mutex;     /* both initialised to 1 */
int   read_count = 0;

void writer(void) {
    sem_wait(&rw_mutex);
    /* ... write shared data ... */
    sem_post(&rw_mutex);
}

void reader(void) {
    sem_wait(&mutex);
    if (++read_count == 1) sem_wait(&rw_mutex);   /* first reader */
    sem_post(&mutex);
```

```
    /* ... read shared data ... */

    sem_wait(&mutex);
    if (--read_count == 0) sem_post(&rw_mutex);   /* last reader  */
    sem_post(&mutex);
}
```



شکل ۵ — فلوچارت خواننده و نویسنده (راه‌حل سمافوری)

نسخهٔ عادلانه با سمافور

افزودن یک سمافور صف `service` که همهٔ درخواست‌ها از آن عبور می‌کنند، ترتیب رسیدن را حفظ می‌کند:

third problem — سودوکد

```
semaphore service = 1, rw_mutex = 1, mutex = 1;  int read_count = 0;


Reader:                              Writer:
  wait(service);                       wait(service);
  wait(mutex);                         wait(rw_mutex);
  if (++read_count == 1)               signal(service);
      wait(rw_mutex);                    // WRITE
  signal(service);                     signal(rw_mutex);
  signal(mutex);
    // READ
  wait(mutex);
  if (--read_count == 0) signal(rw_mutex);
  signal(mutex);
```

راه‌حل با مانیتور (اولویت با نویسنده)

سودوکد

```
monitor ReadersWriters {
    int readers = 0, waitingWriters = 0;  boolean writing = false;
    condition okToRead, okToWrite;

    procedure startRead() {
        while (writing || waitingWriters > 0) okToRead.wait();
        readers++;
    }
    procedure endRead() {
        readers--;
        if (readers == 0) okToWrite.signal();
    }
    procedure startWrite() {
        waitingWriters++;
        while (writing || readers > 0) okToWrite.wait();
        waitingWriters--;  writing = true;
    }
    procedure endWrite() {
        writing = false;
        if (waitingWriters > 0) okToWrite.signal();
        else okToRead.broadcast();
    }
}
```

کد C (pthread monitor)

```c
#include <pthread.h>

static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t okToRead  = PTHREAD_COND_INITIALIZER;
static pthread_cond_t okToWrite = PTHREAD_COND_INITIALIZER;
static int readers = 0, writing = 0, waitingWriters = 0;

void start_read(void) {
    pthread_mutex_lock(&m);
    while (writing || waitingWriters > 0) pthread_cond_wait(&okToRead, &m);
    readers++;
    pthread_mutex_unlock(&m);
}
void end_read(void) {
    pthread_mutex_lock(&m);
    if (--readers == 0) pthread_cond_signal(&okToWrite);
    pthread_mutex_unlock(&m);
}
void start_write(void) {
    pthread_mutex_lock(&m);
    waitingWriters++;
    while (writing || readers > 0) pthread_cond_wait(&okToWrite, &m);
    waitingWriters--;  writing = 1;
    pthread_mutex_unlock(&m);
}
void end_write(void) {
    pthread_mutex_lock(&m);
    writing = 0;
    if (waitingWriters > 0) pthread_cond_signal(&okToWrite);
    else pthread_cond_broadcast(&okToRead);
    pthread_mutex_unlock(&m);
}
```

POSIX خودش هم قفل آمادهٔ خواننده–نویسنده (`pthread_rwlock_t`) دارد که در عمل به‌جای نوشتن دستی استفاده می‌شود.

۷. مقایسهٔ جامع و جدول‌ها

در این بخش پنج روش بر پایهٔ پارامترهای درستی (جدول ۵) و پارامترهای مهندسی (جدول ۶) مقایسه می‌شوند. ستون «خوانندگان–نویسندگان» به راه‌حل سمافوری استاندارد (اولویت با خواننده) اشاره دارد، مگر خلاف آن ذکر شود.

جدول ۵ — مقایسهٔ پارامترهای درستی و رفتار

پارامتر	Dekker	Peterson	Semaphore	Monitor	Readers–Writers
انحصار متقابل	□	□	□ (با استفادهٔ درست)	□ (خودکار)	□ (نویسنده با همه)
پیشرفت (Progress)	□	□	□	□	□
انتظار محدود	□ (حداکثر یک بار)	□ (حداکثر یک بار)	فقط با صف FIFO	معمولاً با صف FIFO	نسخهٔ اول □، سوم □
قحطی (Starvation)	ندارد	ندارد	ممکن (سمافور ضعیف)	ممکن (در مزا)	نویسنده (نسخهٔ ۱) / خواننده (نسخهٔ ۲)
بن‌بست (Deadlock)	ندارد	ندارد	ممکن با ترتیب غلط	ممکن (فراخوانی تو در تو)	ممکن با ترتیب غلط
انتظار مشغول	دارد	دارد	ندارد (نوع مسدودکننده)	ندارد	ندارد
عدالت (Fairness)	تقریباً عادلانه (۲ فرایند)	عادلانه (۲ فرایند)	وابسته به صف	وابسته به صف شرط	نسخهٔ سوم عادلانه
همروندی خواننده‌ها	—	—	—	—	دارد (هدف مسئله)

جدول ۶ — مقایسهٔ پارامترهای مهندسی و کاربردی

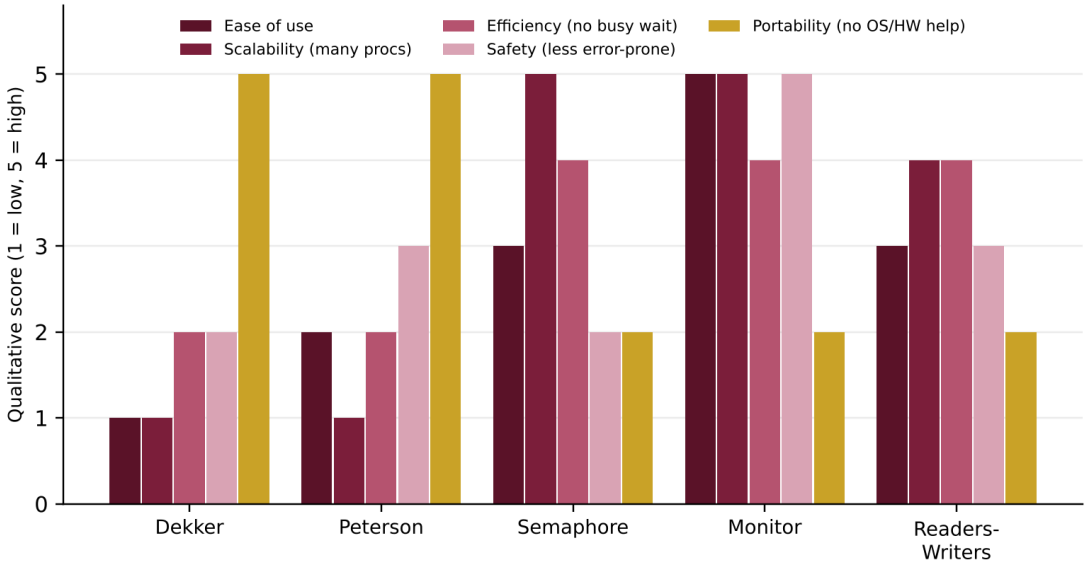
پارامتر	Dekker	Peterson	Semaphore	Monitor	Readers–Writers
سال / مبدع	۱۹۶۵، دکر / دایکسترا	۱۹۸۱، پترسون	۱۹۶۵، دایکسترا	۱۹۷۴، هوآر / برینچ هانسن	۱۹۷۱، کورتوآ و همکاران
نوع	الگوریتم نرم‌افزاری	الگوریتم نرم‌افزاری	ابزار سیستم‌عامل	سازهٔ سطح زبان	مسئلهٔ کلاسیک
تعداد فرایند	۲	۲ (تعمیم: Filter)	N	N	N خواننده + N نویسنده
نیاز به سخت‌افزار / OS	فقط حافظه (+ ترتیب حافظه)	فقط حافظه (+ ترتیب حافظه)	عمل اتمیک + صف OS	قفل + متغیر شرطی	سمافور یا مانیتور
متغیرهای مشترک	۳	۳	۱ (mutex)	۰ صریح (قفل داخلی)	۳ (mutex, rw_mutex, read_count)
سطح انتزاع	پایین	پایین	متوسط	بالا	الگوی طراحی
سهولت استفاده	دشوار	نسبتاً ساده	متوسط	آسان	متوسط
خطاپذیری	زیاد	متوسط	زیاد	کم	متوسط
هزینهٔ اجرا	مصرف CPU بالا	مصرف CPU بالا	تعویض متن هنگام انتظار	قفل + تعویض متن	مانند ابزار پایه
کاربرد معمول	آموزش، نظریه	آموزش، نظریه، سیستم‌های ساده	هسته، نخ‌ها، منابع شمارشی	جاوا، C#، کتابخانه‌ها	پایگاه داده، حافظهٔ نهان

جمع‌بندی جدول‌ها

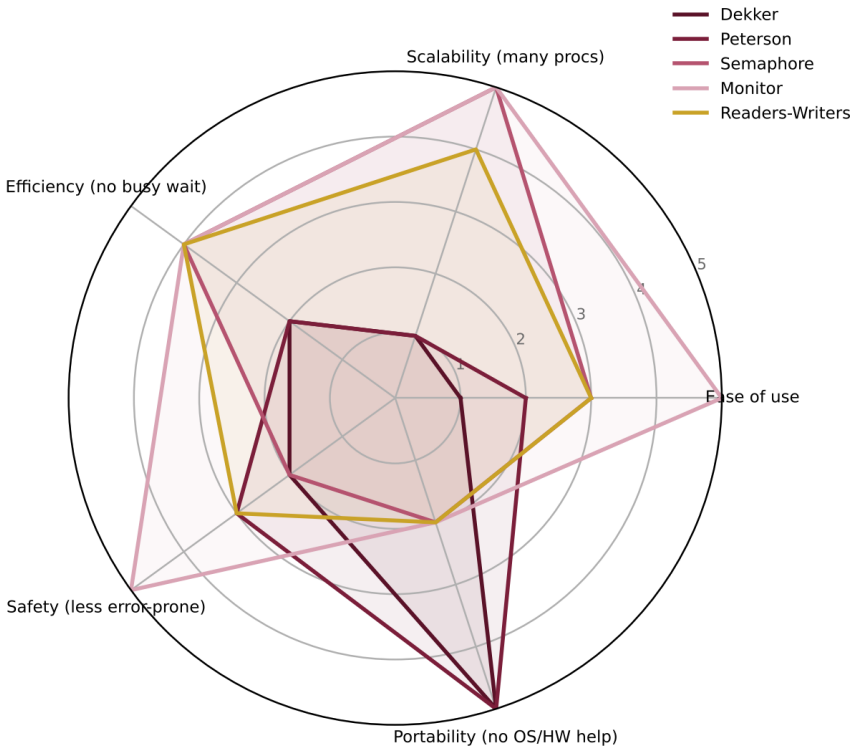
برای **آموزش** و نظریهٔ درستی، دکر و پترسون مناسب‌اند؛ برای **بیش از دو فرایند** و پرهیز از انتظار مشغول، سمافور؛ برای **کد ایمن‌تر و خواناتر**، مانیتور؛ و وقتی خواننده‌ها بسیار و نویسنده‌ها کم‌اند، الگوی خوانندگان–نویسندگان با انتخاب سیاست مناسب (اولویت خواننده، نویسنده یا عادلانه).

۸. نمودارها

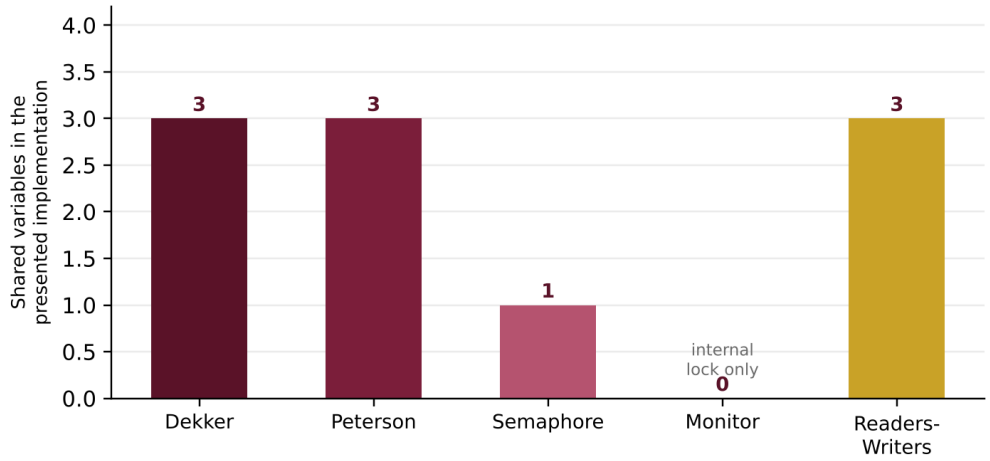
نمودارهای زیر چند جنبهٔ مقایسه را بصری می‌کنند. امتیازهای نمودارهای ۱ و ۲ **ارزیابی کیفی نگارنده** بر پایهٔ ویژگی‌های ساختاری روش‌هاست (۱ = ضعیف، ۵ = قوی) و حاصل اندازه‌گیری تجربی نیستند.



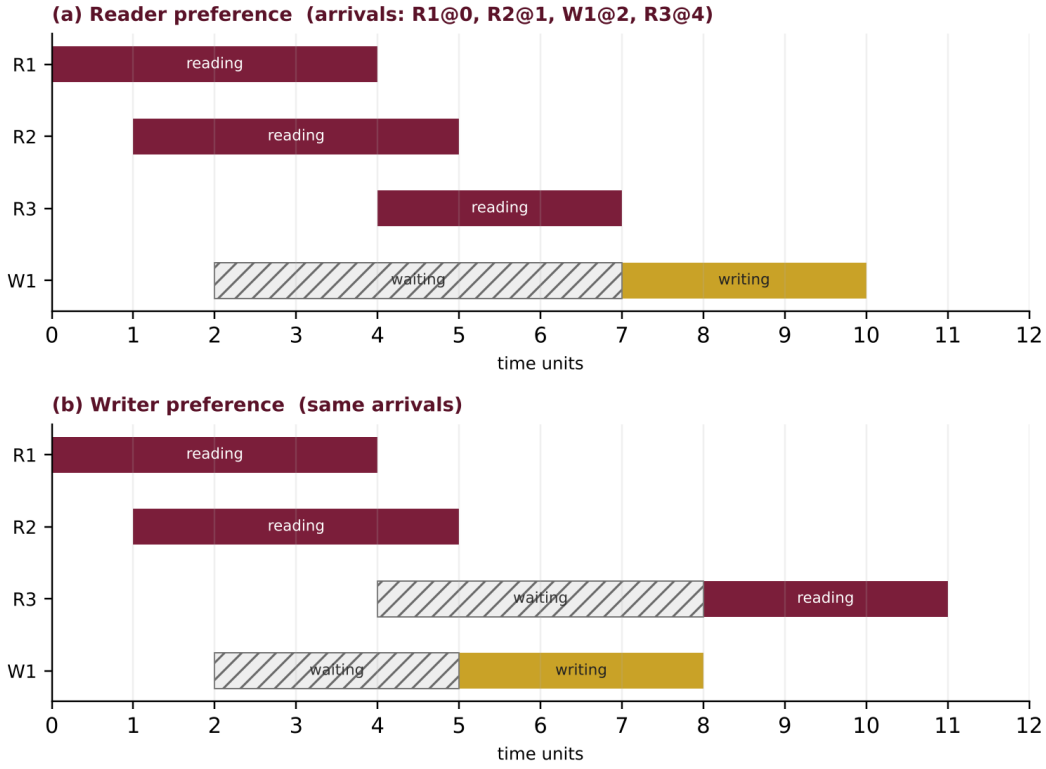
شکل ۶ — مقایسهٔ کیفی روش‌ها در پنج معیار (نمودار میله‌ای)



شکل ۷ — نمودار راداری همان معیارها



شکل ۸ — تعداد متغیرهای مشترک در پیاده‌سازی‌های ارائه‌شده



شکل ۹ — نمودار زمانی نمونه: اولویت خواننده در برابر اولویت نویسنده

در شکل ۹ (یک سناریوی مثالی، نه اندازه‌گیری): `R1` در زمان ۰، `R2` در ۱، `W1` در ۲ و `R3` در ۴ می‌رسند. با اولویت خواننده، `R3` وارد می‌شود چون خواننده‌ها در حال خواندن‌اند و `W1` تا زمان ۷ منتظر می‌ماند. با اولویت نویسنده، `R3` پشت `W1` می‌ماند و `W1` پس از پایان `R1` و `R2` (زمان ۵) می‌نویسد.

۹. مثال‌ها و نمونه‌سؤال‌ها

برای هر یک از پنج روش، یک مثال حل‌شده و چند سؤال همراه با پاسخ آمده است.

۹-۱. دکر

مثال حل‌شده

اگر `turn = 1` باشد و فقط `P0` بخواهد وارد شود چه می‌شود؟ `P0` مقدار `flag[0]=true` را می‌نویسد، `flag[1]` را نادرست می‌بیند و **بلافاصله** وارد می‌شود، با اینکه نوبت با `P1` است. این همان شرط پیشرفت است: متغیر `turn` فقط هنگام تعارض اهمیت دارد.

سؤال ۱: اگر در الگوریتم دکر دستور `flag[i] = false` درون حلقه حذف شود چه رخ می‌دهد؟

پاسخ: بن‌بست پیش می‌آید. فرض کنید `turn = 0` و هر دو پرچم درست است. `P1` چون `turn ≠ 1` است در انتظار `turn == 1` می‌ماند ولی پرچمش درست می‌ماند؛ `P0` پرچم `P1` را درست می‌بیند و در حلقهٔ `while` می‌چرخد. `turn` فقط پس از خروج `P0` عوض می‌شود، پس هیچ‌کدام پیش نمی‌رود.

سؤال ۲: دکر حداکثر برای چند فرایند کار می‌کند و آیا انتظار محدود دارد؟

پاسخ: فقط ۲ فرایند. بله، انتظار محدود دارد: پس از خروج، `turn` به رقیب داده می‌شود و هر فرایند حداکثر یک بار منتظر ورود رقیب می‌ماند.

۹-۲. پترسون

مثال حل‌شده

اگر `P1` در بخش باقیمانده باشد، `flag[1]=false` است. `P0` پرچم و `turn=1` را می‌نویسد، اما شرط `flag[1] && turn==1` نادرست است و وارد می‌شود: متقاضی تنها هرگز مسدود نمی‌شود.

سؤال ۳: اگر ترتیب دو دستور ورود عوض شود (ابتدا `turn = j` سپس `flag[i] = true`) انحصار متقابل نقض می‌شود؟ سناریو بسازید.

پاسخ: بله. `P0: turn=1` ؛ `P1: turn=0` ؛ `P1: flag[1]=true` ؛ `P1` پرچم `flag[0]` را نادرست می‌بیند و وارد می‌شود. سپس `P0: flag[0]=true` ؛ شرط `flag[1] && turn==1` نادرست است چون `turn==0` ؛ پس `P0` هم وارد می‌شود و هر دو درون ناحیهٔ بحرانی‌اند.

سؤال ٤: چرا پترسون روی پردازنده‌های مدرن بدون حصار حافظه ممکن است خراب شود؟

پاسخ: پردازنده یا کامپایلر می‌تواند ترتیب نوشتن `flag[i]=true` و خواندن `flag[j]` را عوض کند (به‌ویژه با بافر نوشتن). در نتیجه هر دو ممکن است پرچم دیگری را نادرست بخوانند و وارد شوند. راه‌حل: متغیرهای اتمیک `seq_cst` یا حصار حافظه.

۹-۳. سمافور

مثال حل‌شده — سمافور شمارنده با مقدار اولیه ۲

چهار فرایند `A..D` به‌ترتیب `wait(S)` می‌زنند، سپس `A` و `B` `signal(S)` می‌زنند:

`A: S=1` ؛ `B: S=0` ؛ `C: S=-1` (مسدود) ؛ `D: S=-2` (مسدود) ؛ `A: signal → S=-1` و `C` بیدار می‌شود ؛ `B: signal → S=0` و `D` بیدار می‌شود. مقدار منفی تعداد منتظران را نشان می‌دهد.

سؤال ۵: با سمافور تضمین کنید که دستور `S1` از `P1` همیشه پیش از `S2` از `P2` اجرا شود.

پاسخ: سمافور `sync` را با مقدار ۰ بسازید. `P1: S1; signal(sync);` و `P2: wait(sync); S2;`. اگر `P2` زودتر برسد مسدود می‌شود تا `P1` سیگنال دهد.

سؤال ۶: نشان دهید چگونه ترتیب اشتباه `wait` بن‌بست می‌سازد.

پاسخ: با دو سمافور `A=1` و `B=1`: `P1: wait(A); wait(B);` و `P2: wait(B); wait(A);`. اگر `P1` سمافور `A` و `P2` سمافور `B` را بگیرد، هر یک منتظر دیگری می‌ماند. راه‌حل: ترتیب ثابت گرفتن سمافورها.

۹-۴. مانیتور

مثال حل‌شده — بافر با ظرفیت ۲

`put(x)` ، `put(y)` پر می‌کنند (`count=2`). `put(z)` در `notFull.wait()` مسدود می‌شود و قفل را آزاد می‌کند. `get()` عنصر را برمی‌دارد، `t=1coun` می‌شود و `notFull.signal()` تولیدکنندهٔ منتظر را بیدار می‌کند؛ او شرط `count==N` را دوباره بررسی می‌کند، نادرست می‌یابد و `z` را درج می‌کند.

سؤال ۷: چرا در مانیتور مزا به‌جای `if` باید از `while` برای انتظار روی شرط استفاده کرد؟

پاسخ: چون بین `signal` و دوباره‌گرفتن قفل ممکن است فرایند دیگری وارد شود و شرط را تغییر دهد (یا بیدارشدن کاذب رخ دهد). `while` شرط را پس از بیدار شدن دوباره بررسی می‌کند.

سؤال ۸: تفاوت `signal` سمافور و `signal` متغیر شرطی چیست؟

پاسخ: `signal` سمافور مقدار شمارنده را زیاد می‌کند و **به یاد سپرده می‌شود**؛ اما `signal` متغیر شرطی در نبود منتظر **بی‌اثر** است و گم می‌شود.

۹-۵. خوانندگان و نویسندگان

مثال حل‌شده — اولویت با خواننده

`R1` وارد می‌شود (`read_count=1`، `rw_mutex` را می‌گیرد). `W1` روی `wait(rw_mutex)` مسدود می‌شود. `R2` می‌رسد: `read_count=2` و چون ۱ نیست `rw_mutex` را نمی‌خواهد، پس وارد می‌شود. تا زمانی که خواننده‌ها یکی پس از دیگری برسند، `read_count` صفر نمی‌شود و `W1` منتظر می‌ماند؛ این همان قحطی نویسنده است.

سؤال ۹: نقش `mutex` و `rw_mutex` در راه‌حل سمافوری چیست؟

پاسخ: `mutex` فقط از متغیر `read_count` در برابر دسترسی همزمان خواننده‌ها محافظت می‌کند. `rw_mutex` دسترسی به داده را کنترل می‌کند: نویسنده آن را کامل می‌گیرد و خواننده‌ها آن را به‌صورت گروهی (با نخستین خواننده می‌گیرند و با آخرین رها می‌کنند).

سؤال ۱۰: کدام نسخه‌ها دچار قحطی می‌شوند و چگونه می‌توان آن را برطرف کرد؟

پاسخ: نسخهٔ اول قحطی نویسنده و نسخهٔ دوم قحطی خواننده دارد. نسخهٔ سوم با صف سرویس (سمافور `service`) یا صف FIFO در مانیتور ترتیب رسیدن را حفظ می‌کند و قحطی ندارد.

سؤال ۱۱: (تستی) کدام روش هم برای بیش از دو فرایند کار می‌کند و هم انتظار مشغول ندارد؟ الف) دکر ب) پترسون ج) سمافور مسدودکننده د) هیچ‌کدام

پاسخ: گزینهٔ **ج**؛ دکر و پترسون فقط دو فرایند را پشتیبانی می‌کنند و انتظار مشغول دارند.

۱۰. نتیجه‌گیری

دکر و پترسون نشان می‌دهند که انحصار متقابل را می‌توان تنها با حافظهٔ مشترک و بدون دستور ویژه ساخت، اما این راه‌حل‌ها به دو فرایند محدودند، انتظار مشغول دارند و به ترتیب حافظهٔ درست وابسته‌اند. سمافور مقیاس‌پذیر و کارآمد است ولی ابزاری سطح پایین و خطاپذیر است. مانیتور با ساختار و انحصار خودکار، برنامه‌نویسی همگام را ایمن‌تر می‌کند و در زبان‌های امروزی بسیار رایج است. مسئلهٔ خوانندگان و نویسندگان نیز نشان می‌دهد که انتخاب **سیاست** (اولویت خواننده، نویسنده یا عدالت) به‌اندازهٔ انتخاب ابزار اهمیت دارد.

- برای یادگیری مفاهیم درستی: دکر و پترسون.
- برای کنترل منابع و ترتیب اجرا میان چند فرایند: سمافور.
- برای کد ساخت‌یافته و کم‌خطا: مانیتور (یا قفل + متغیر شرطی).
- برای دادهٔ مشترک با خواندن زیاد: الگوی خوانندگان–نویسندگان یا `pthread_rwlock_t`.

۱۱. منابع

- Dijkstra, E. W. (۱۹٦٥). Cooperating Sequential Processes (EWD۱۲۳). Technological University Eindhoven.
- Peterson, G. L. (۱۹۸۱). Myths About the Mutual Exclusion Problem. Information Processing Letters, ۱۲(۳), ۱۱٥–۱۱٦.
- Hoare, C. A. R. (۱۹۷٤). Monitors: An Operating System Structuring Concept. Communications of the ACM, ۱۷(۱۰), ٥٤۹–٥٥۷.
- Brinch Hansen, P. (۱۹۷۳). Operating System Principles. Prentice Hall.
- Courtois, P. J., Heymans, F., & Parnas, D. L. (۱۹۷۱). Concurrent Control with “Readers” and “Writers”. Communications of the ACM, ۱٤(۱۰), ٦٦۷–٦٦۸.
- Silberschatz, A., Galvin, P. B., & Gagne, G. Operating System Concepts. Wiley.
- Tanenbaum, A. S., & Bos, H. Modern Operating Systems. Pearson.